JAVA PROGRAMMING / DSA

Chapter 47: Number Theory Basics

Personal Notes

Number theory sounds abstract, but it's everywhere in coding interviews. Modular arithmetic prevents overflow when returning huge answers. GCD tricks compress fractions and reveal cycle lengths. The Sieve of Eratosthenes generates primes in O(n log log n). Modular exponentiation computes 2^100000 mod p in microseconds. These aren't math trivia — they're workhorse tools for a specific class of interview problems.

This chapter covers the four practical corners of number theory for DSA: GCD/LCM (Euclidean algorithm), primes and sieve, prime factorization, and modular arithmetic (including fast power and inverse). Together they handle almost every number-theory question you'll see in a coding interview, plus dozens of related DP and combinatorics problems that use mod 10^9+7.

1. Why Number Theory Matters

You'll reach for number theory whenever a problem involves any of these:

- "Return the answer MOD $10^9 + 7$" (any answer that could overflow long)
- GCD / LCM computation, reducing fractions
- Enumerating primes up to N, checking primality
- Counting divisors or prime factors
- Computing large powers modulo something
- nCr (combinations) with modular arithmetic
- Cycle detection in modular sequences (Floyd's, cryptography)

> **The mod $10^9 + 7$ pattern:** Whenever a competitive-programming problem could return numbers too big for int or long, the expected answer is computed modulo a large prime — usually $10^9 + 7$ or 998244353. All arithmetic (add, multiply, exponent) is done under mod. Getting this right requires the tools in this chapter.

2. GCD and LCM

The GREATEST COMMON DIVISOR (GCD) of a and b is the largest integer that divides both. The LEAST COMMON MULTIPLE (LCM) is the smallest positive integer that's a multiple of both. These are the two foundational number-theory operations.

2.1 Euclidean Algorithm — GCD in O(log min(a, b))

The classical trick: gcd(a, b) = gcd(b, a mod b). Apply repeatedly until one becomes 0.

```
public int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

// Recursive one-liner:
public int gcdRec(int a, int b) {
    return b == 0 ? a : gcdRec(b, a % b);
}

// Time: O(log min(a, b)) — Fibonacci is the worst case
```

LCM via GCD

```
public long lcm(long a, long b) {
    return a / gcd((int)a, (int)b) * b;
    // Note: divide FIRST to avoid overflow, THEN multiply
}
```

The a / gcd() * b order matters: Doing (a * b) / gcd(a, b) can overflow long even when the final answer fits. Divide by gcd BEFORE multiplying — a/gcd is guaranteed to divide a exactly, so we don't lose precision, and the intermediate result stays smaller.

Dry Run for gcd(48, 18)

```
Step 1:  a=48, b=18   →  48 mod 18 = 12    →  a=18, b=12
Step 2:  a=18, b=12   →  18 mod 12 = 6     →  a=12, b=6
Step 3:  a=12, b=6    →  12 mod 6  = 0     →  a=6,  b=0
Result: gcd = 6

Only 3 steps — far fewer than examining every divisor of 48 and 18.
```

3. Extended Euclidean Algorithm

The extended version finds not just gcd(a, b) but also integers x and y such that a*x + b*y = gcd(a, b). Used to compute modular inverses when the modulus isn't prime. Rarely needed in interviews but worth knowing about.

```
public int[] extGcd(int a, int b) {
    if (b == 0) return new int[]{a, 1, 0};   // gcd, x, y
    int[] r = extGcd(b, a % b);
    return new int[]{r[0], r[2], r[1] - (a / b) * r[2]};
}
```

4. Primes and the Sieve of Eratosthenes

A PRIME NUMBER is an integer > 1 with no divisors other than 1 and itself. To enumerate all primes up to N, the SIEVE OF ERATOSTHENES is the standard tool — O(N log log N), efficient enough for N up to 10^7.

4.1 Primality Check (Single Number)

```
public boolean isPrime(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int i = 3; (long) i * i <= n; i += 2) {
        if (n % i == 0) return false;
    }
    return true;
}

// Time: O(sqrt(n))
```

Why check up to sqrt(n): If n has a divisor > sqrt(n), it must also have a corresponding divisor < sqrt(n). So checking up to sqrt(n) catches at least one of each divisor pair. This trick cuts the check from O(n) to O(sqrt n) — a huge win.

4.2 Sieve of Eratosthenes

To find ALL primes up to N: mark every multiple of every prime as non-prime, starting from 2.

```
public boolean[] sieve(int n) {
    boolean[] isPrime = new boolean[n + 1];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;

    for (int i = 2; (long) i * i <= n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= n; j += i) {
                isPrime[j] = false;
            }
        }
    }
    return isPrime;
}

// Time: O(n log log n)
// Space: O(n)
```

Why Start Marking from i*i

For prime i, all multiples less than i*i have already been marked by SMALLER primes (2, 3, ..., i-1). E.g., when i=5, multiples 10, 15, 20 are already marked by 2 or 3. So we can start at $i^2 = 25$.

```
     Sieve up to n = 30:

     Start:  2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24
25 26 27 28 29 30

     Mark multiples of 2 (from 4):   __ 3 __ 5 __ 7 __ 9 __ 11 __ 13 __ 15
__ 17 __ 19 __ 21 __ 23 __ 25 __ 27 __ 29 __
     Mark multiples of 3 (from 9):   __ 3 __ 5 __ 7 __ __ __ 11 __ 13 __
__ __ 17 __ 19 __ __ __ 23 __ 25 __ __ __ 29 __
     Mark multiples of 5 (from 25):  __ 3 __ 5 __ 7 __ __ __ 11 __ 13 __
__ __ 17 __ 19 __ __ __ 23 __ __ __ __ __ 29 __

     Primes ≤ 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29
```

5. Prime Factorization

Every integer > 1 has a UNIQUE factorization into primes. Finding it is O(sqrt n) for a single number — divide by each prime up to sqrt(n) as many times as possible, then whatever remains (if > 1) is a prime factor.

```
public List<Integer> primeFactors(int n) {
    List<Integer> factors = new ArrayList<>();
    for (int i = 2; (long) i * i <= n; i++) {
        while (n % i == 0) {
            factors.add(i);
            n /= i;
        }
    }
    if (n > 1) factors.add(n);   // remaining prime > sqrt(original)
    return factors;
}

// primeFactors(60) → [2, 2, 3, 5]  (60 = 2² × 3 × 5)
// Time: O(sqrt n)
```

Fast Factorization Using a Sieve

If you'll factorize MANY numbers up to N, precompute the smallest prime factor (SPF) for each — then factorization is O(log n) per number.

```
public int[] smallestPrimeFactor(int n) {
    int[] spf = new int[n + 1];
    for (int i = 2; i <= n; i++) {
        if (spf[i] == 0) {              // i is prime
            for (int j = i; j <= n; j += i) {
```

```
                if (spf[j] == 0) spf[j] = i;
            }
        }
    }
    return spf;
}

public List<Integer> fastFactor(int n, int[] spf) {
    List<Integer> factors = new ArrayList<>();
    while (n > 1) {
        factors.add(spf[n]);
        n /= spf[n];
    }
    return factors;
}
```

6. Modular Arithmetic — The Foundation

Modular arithmetic answers: what's the remainder when we do arithmetic and then divide by M? Under mod M, addition and multiplication work "like normal" — but with a wraparound.

Basic Rules

```
     (a + b) mod M = ((a mod M) + (b mod M)) mod M
     (a - b) mod M = ((a mod M) - (b mod M) + M) mod M    ← the + M keeps
result non-negative
     (a * b) mod M = ((a mod M) * (b mod M)) mod M
     (a^n) mod M   = uses modular exponentiation (next section)

     DIVISION IS DIFFERENT — needs modular inverse (also below).
```

The Standard Practice

```
long MOD = 1_000_000_007L;

// Add
long sum = (a + b) % MOD;

// Multiply — cast to long to avoid overflow
long prod = ((long) a * b) % MOD;

// Subtract — add MOD first to keep positive
long diff = ((a - b) % MOD + MOD) % MOD;
```

The overflow trap: In Java, int * int overflows silently and stays int. Always CAST to long before multiplying: ((long)a * b) % MOD, not (a * b) % MOD. Missing this cast is the #1 bug in modular arithmetic code.

7. Modular Exponentiation — Fast Power

Computing a^n mod m by multiplying a n times is O(n) — way too slow for n = 10^9. FAST EXPONENTIATION uses squaring to do it in O(log n) — the workhorse of modular exponentiation, cryptography, and many DP problems.

The Idea

```
     a^13 = a^(8+4+1)
          = a^8 × a^4 × a^1
          = (a^4)² × (a^2)² × a
          = (((a²)²)²) × ((a²)²) × a

     Every "×2" step is one square. The bit pattern of n (13 = 1101₂)
     tells us which squares to include in the final product.
```

```
public long modPow(long base, long exp, long mod) {
    long result = 1;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) {
            result = (result * base) % mod;
        }
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}

// Time: O(log exp)
// Space: O(1)

// Example: 2^10 mod 1000 = 24
//   exp bits: 1010 (10)
//   Loop through bits, squaring base each time
```

This is one of the most important algorithms in the chapter: Modular exponentiation is used in RSA cryptography, primality tests, computing large Fibonacci numbers, and matrix exponentiation for advanced DP. If you memorize one number-theory algorithm, make it this one.

8. Modular Inverse

Regular division doesn't work under modulo — (a/b) mod M is NOT (a mod M) / (b mod M). Instead, we multiply by the MODULAR INVERSE of b: the value b^{-1} such that $b \times b^{-1} \equiv 1 \pmod M$.

Fermat's Little Theorem (When M is prime)

If M is prime and gcd(a, M) = 1, then a^(M-1) ≡ 1 (mod M). Therefore a^(M-2) ≡ a^{-1} (mod M). That gives a clean O(log M) inverse formula using fast exponentiation.

```
public long modInverse(long a, long mod) {
    return modPow(a, mod - 2, mod);   // Works only when mod is PRIME
}

// Example: inverse of 3 mod 11
//   modInverse(3, 11) = 3^9 mod 11 = 4
//   Verify: 3 × 4 = 12 ≡ 1 (mod 11) ✓

// For non-prime mod, use the extended Euclidean algorithm instead.
```

Division Under Modulo

```
// (a / b) mod M when M is prime:
long divide = (a * modInverse(b, M)) % M;
```

> **$10^9 + 7$ is prime:** That's why it's chosen for competitive programming problems — Fermat's little theorem works cleanly for computing inverses. If a problem uses a non-prime modulus, use the extended Euclidean algorithm instead.

9. Combinatorics — nCr Under Mod

"How many ways to choose k items from n?" — nCr = n! / (k! × (n-k)!). For huge n, we can't compute the raw factorial (overflow), so we work mod M throughout using precomputed factorial and inverse-factorial arrays.

```
class Combinatorics {
    long MOD = 1_000_000_007L;
    long[] fact, invFact;

    public Combinatorics(int maxN) {
        fact = new long[maxN + 1];
        invFact = new long[maxN + 1];
        fact[0] = 1;
        for (int i = 1; i <= maxN; i++) {
            fact[i] = fact[i - 1] * i % MOD;
        }
        invFact[maxN] = modPow(fact[maxN], MOD - 2, MOD);
        for (int i = maxN - 1; i >= 0; i--) {
            invFact[i] = invFact[i + 1] * (i + 1) % MOD;
        }
    }

    public long nCr(int n, int r) {
        if (r < 0 || r > n) return 0;
        return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
    }

    private long modPow(long b, long e, long m) {
```

```
        long r = 1;
        b %= m;
        while (e > 0) {
            if ((e & 1) == 1) r = r * b % m;
            b = b * b % m;
            e >>= 1;
        }
        return r;
    }
}

// Precompute once — then each nCr(n, r) is O(1).
```

The invFact backward pass: Computing invFact[maxN] once with modPow, then filling invFact[i] = invFact[i+1] * (i+1) is faster than calling modPow(fact[i], …) for every i. Same math result, but O(n) instead of O(n log n).

10. Classic Problem 1 — Count Primes

LeetCode 204. Count the number of prime numbers less than n. Direct application of the Sieve of Eratosthenes.

```
public int countPrimes(int n) {
    if (n <= 2) return 0;
    boolean[] isPrime = new boolean[n];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;

    for (int i = 2; (long) i * i < n; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j < n; j += i) {
                isPrime[j] = false;
            }
        }
    }

    int count = 0;
    for (boolean prime : isPrime) if (prime) count++;
    return count;
}

// Time: O(n log log n)
```

11. Classic Problem 2 — GCD of Strings

LeetCode 1071. Given two strings, find the largest string that DIVIDES both (via concatenation). Trick: if the strings share a valid "gcd string," then str1 + str2 = str2 + str1 — and the answer's length is gcd(len1, len2).

```
public String gcdOfStrings(String str1, String str2) {
    if (!(str1 + str2).equals(str2 + str1)) return "";
    int g = gcd(str1.length(), str2.length());
    return str1.substring(0, g);
}

private int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

Beautiful GCD application — you'd never guess string concatenation had a GCD analog until you see the trick.

12. Classic Problem 3 — Super Pow

LeetCode 372. Compute a^b mod 1337 where b is given as an array of digits (b can be arbitrarily huge). Modular exponentiation on steroids.

```
public int superPow(int a, int[] b) {
    long result = 1;
    long base = a % 1337;
    // Process digits from most significant to least
    for (int digit : b) {
        result = (modPow(result, 10, 1337) * modPow(base, digit, 1337)) %
1337;
    }
    return (int) result;
}

private long modPow(long base, long exp, long mod) {
    long r = 1;
    base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) r = r * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return r;
}
```

> **The trick — treat b as digits:** a^123 = a^(120+3) = a^120 × a^3 = (a^12)^10 × a^3. So we can process b one digit at a time: at each step, raise the running result to the 10th power and multiply by a^digit. Standard fast exponentiation handles the rest.

13. Classic Problem 4 — Number of Ways to Reach N-th Stair (mod 10^9+7)

Standard DP problem — but with a huge N. The recurrence is Fibonacci-like; the answer must be returned mod 10^9+7.

```
public int climbStairs(int n) {
    long MOD = 1_000_000_007L;
    long[] dp = new long[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = (dp[i - 1] + dp[i - 2]) % MOD;
    }
    return (int) dp[n];
}
```

For truly astronomical N (10^{18}), you'd use MATRIX EXPONENTIATION — express the recurrence as a 2×2 matrix and use fast exponentiation on matrices. Same O(log n) technique, different data.

14. Classic Problem 5 — Perfect Squares (Sum of Squares)

LeetCode 279. Given n, return the least number of perfect squares that sum to n. LAGRANGE'S FOUR-SQUARE THEOREM guarantees the answer is always 1, 2, 3, or 4. LEGENDRE'S THREE-SQUARE THEOREM tells us EXACTLY when it's 4: n = 4^a × (8b + 7).

```
public int numSquares(int n) {
    // Check if n is a perfect square (answer = 1)
    int sqrtN = (int) Math.sqrt(n);
    if (sqrtN * sqrtN == n) return 1;

    // Check if n = 4^a × (8b + 7)  →  answer = 4
    int temp = n;
    while (temp % 4 == 0) temp /= 4;
    if (temp % 8 == 7) return 4;

    // Check if n is a sum of two squares (answer = 2)
    for (int i = 1; i * i <= n; i++) {
        int rem = n - i * i;
        int rSqrt = (int) Math.sqrt(rem);
        if (rSqrt * rSqrt == rem) return 2;
    }
```

```
    // Otherwise 3
    return 3;
}

// Time: O(sqrt n) — MUCH faster than the DP version
```

When math beats DP: The standard DP solution runs in O(n × sqrt n). This number-theoretic version is O(sqrt n) — a dramatic speedup driven by two theorems from classical number theory. Real reminder that sometimes the fast algorithm needs pure math, not more code.

15. When to Reach for Each Technique

If you need to...	Use
Compute gcd or lcm of two numbers	**Euclidean algorithm**
Check if a number is prime (single check)	**Trial division up to sqrt(n)**
Generate all primes up to N	**Sieve of Eratosthenes**
Prime factorization of one number	**Trial division up to sqrt(n)**
Prime factorization of MANY numbers ≤ N	**Smallest-prime-factor sieve**
Compute a^n mod m for huge n	**Fast modular exponentiation**
Compute nCr mod prime	**Precomputed factorials + Fermat inverse**
Divide under modulo (mod is prime)	**Multiply by modular inverse (Fermat)**
Divide under modulo (mod is NOT prime)	**Extended Euclidean algorithm**
Detect "sum-of-squares" structure	**Lagrange/Legendre theorems**
Handle overflow when returning huge counts	**All arithmetic under mod 10^9+7**

16. Complexity Summary

Operation	Time	Space
GCD (Euclidean)	O(log min(a, b))	O(1)
LCM	O(log min(a, b))	O(1)
Primality (trial division)	O(sqrt n)	O(1)
Sieve of Eratosthenes	O(n log log n)	O(n)

Operation	Time	Space
Prime factorization	O(sqrt n)	O(log n)
Fast modular exponent	O(log exp)	O(1)
Modular inverse (Fermat)	O(log mod)	O(1)
nCr with precomputed factorials	O(n) preproc + O(1) per query	O(n)

17. Common Mistakes

Mistake 1: Overflow in modular multiplication

```
long result = (a * b) % MOD;              // ✗ if a, b are int, product
overflows int
long result = ((long) a * b) % MOD;       // ✓ correct — cast one to long
first
```

Mistake 2: Negative modulo

```
long diff = (a - b) % MOD;                    // ✗ can be negative
long diff = ((a - b) % MOD + MOD) % MOD;     // ✓ always non-negative
```

Mistake 3: Using Fermat with non-prime mod

modPow(a, mod-2, mod) computes the inverse ONLY when mod is prime. For composite mod, use the extended Euclidean algorithm. Using Fermat's incorrectly returns wrong values silently.

Mistake 4: Off-by-one in sieve boundaries

```
for (int i = 2; i * i <= n; i++)          // ✗ i * i can overflow int
for (int i = 2; (long) i * i <= n; i++)   // ✓ safe
```

Mistake 5: Starting sieve inner loop at 2*i instead of i*i

Both work correctness-wise, but i*i is faster — multiples smaller than i*i are already marked by smaller primes.

Mistake 6: Computing LCM as (a * b) / gcd(a, b)

```
long lcm = (a * b) / gcd(a, b);       // ✗ a * b can overflow long
long lcm = a / gcd(a, b) * b;         // ✓ divide first, then multiply
```

Mistake 7: Forgetting to precompute factorials

Computing nCr(n, r) from scratch each time is O(n). For thousands of queries, precomputing factorial + inverse factorial arrays makes each nCr call O(1). Skipping this optimization means TLE on large inputs.

18. Best Practices

- Always use long for modular arithmetic to avoid overflow
- Cast int × int to long BEFORE multiplying: ((long) a * b) % MOD
- For subtraction under mod, always add MOD to keep results positive
- Use MOD = 1_000_000_007L (prime) so Fermat's inverse works
- Sieve for many primality checks; trial division for a single one
- Compute LCM as a / gcd × b (not (a×b)/gcd) to prevent overflow
- Precompute factorial and invFactorial arrays for many nCr queries
- For number-theory DP with large n, look up matrix exponentiation techniques

19. Quick Reference

Concept	Formula / Meaning
GCD	`gcd(a, b) = gcd(b, a mod b)`
LCM	`a / gcd(a, b) * b`
Prime check	`Trial division up to sqrt(n)`
Sieve	`O(n log log n) prime generator`
Prime factorization	`Trial divide by primes ≤ sqrt(n)`
Modular add	`(a + b) % MOD`
Modular subtract	`((a - b) % MOD + MOD) % MOD`
Modular multiply	`((long) a * b) % MOD`
Modular exponent	`Fast power in O(log exp)`
Modular inverse	`a^(MOD - 2) if MOD is prime (Fermat)`
nCr mod prime	`Precomputed fact[] and invFact[]`
$10^9 + 7$	`Common prime modulus — Fermat works`

20. Practice Problems

Try in order — foundational GCD/prime problems first, then modular arithmetic.

GCD / LCM

1. Greatest Common Divisor of Strings (LeetCode 1071).
2. Nth Magical Number (LeetCode 878) — LCM + binary search.
3. Fraction Addition and Subtraction (LeetCode 592) — GCD-based simplification.
4. Number of Different Subsequences GCDs (LeetCode 1819).

Primes / Sieve

5. Count Primes (LeetCode 204) — Sieve of Eratosthenes.
6. Ugly Number (LeetCode 263).
7. Prime Palindrome (LeetCode 866).
8. Closest Prime Numbers in Range (LeetCode 2523).

Prime Factorization

9. Distinct Prime Factors of Product of Array (LeetCode 2521).
10. Nth Ugly Number (LeetCode 264) — heap or DP.
11. Super Ugly Number (LeetCode 313).

Modular Arithmetic & Fast Power

12. Pow(x, n) (LeetCode 50) — fast exponentiation.
13. Super Pow (LeetCode 372) — huge exponent as digit array.
14. Count Ways to Build Good Strings (LeetCode 2466) — DP mod 10^9+7.
15. Number of Ways to Reach a Position After Exactly K Steps (LeetCode 2400) — nCr mod prime.

Combinatorics / Advanced

16. Number of Combinations (LeetCode 79) — precompute nCr.
17. Perfect Squares (LeetCode 279) — Lagrange four-square shortcut.
18. Fibonacci Number (LeetCode 509) — matrix exponentiation for large n.

21. Final Takeaway

Number theory feels arcane at first — but the practical toolkit for coding interviews is small and highly reusable. Master GCD/LCM, primality/sieve, prime factorization, and modular arithmetic (including fast power and inverse), and you've covered 95% of what interview problems throw at you. The tools compose beautifully — nCr mod prime combines factorials + fast power + Fermat's inverse into one clean O(1) query.

For interviews, master three algorithms: GCD (Euclidean), Sieve of Eratosthenes, and Fast Modular Exponentiation. Add the standard mod tricks (cast to long, +MOD for subtraction) and the nCr precomputation pattern. That's enough for any "answer modulo 10^9+7" DP problem you'll ever see.

> **Key Takeaway:** GCD via Euclidean is O(log min). Sieve gives all primes ≤ N in O(N log log N). Prime factorization is O(sqrt n) — precompute smallest-prime-factor for repeated use. Modular arithmetic requires long casts and +MOD for subtraction. Fast power computes a^n mod m in O(log n) — used for everything from Fermat's inverse to matrix exponentiation. nCr mod prime = fact × invFact × invFact using precomputed arrays. These few tools unlock the entire "answer mod 10^9+7" category.

22. What's Next?

With number theory covered, the classical DSA + core CS toolkit is essentially COMPLETE. Remaining specialized topics:

- Randomized Algorithms — Quickselect, Reservoir Sampling, Monte Carlo methods
- Max Flow / Min Cut — Ford-Fulkerson, Edmonds-Karp
- Computational Geometry — convex hull, line sweep, closest pair
- Advanced DP — knapsack variants, digit DP, matrix exponentiation
- Suffix Arrays / Automata — advanced string data structures
- System Design fundamentals — the natural big step for senior interviews
- Behavioral / Leadership interviews — the missing half of Google prep